Azerbaijan University of Architecture and Construction
ISSN 2706-7726
Engineering Mechanics
Scientific and Technical Journal
E-mail: engineeringmechanics@azmiu.edu.az

June 2025	Issue 20	Volume 12	Number 5	Pages 12-17

APPLICATION OF NEURAL NETWORKS FOR ROBOTICS CONTROL ON THE RP2040 MICROCONTROLLER USING LITERT

NURU DASHDAMIRLI
Azerbaijan Technical University

Abstract: Bringing artificial intelligence (AI) to embedded devices is a significant step forward for robotics, as the frontier of AI is moving from large data centers to small everyday devices with TinyML. This study investigates the practical implementation of neural networks for robotics control on the Raspberry Pi RP2040, a popular low-cost microcontroller. The work demonstrates how to bridge the gap between AI model development and on-device execution by using the LiteRT technology, a runtime environment specifically designed for hardware platforms with limited resources. The integration of the LiteRT platform enables embedded systems to achieve real-time inference. The work presents a workflow for successfully running neural networks for a real-time autonomous navigation system in a simple two-wheeled robot with an ultrasonic sensor mounted on a servo. The findings confirm that the RP2040 is a highly effective platform for edge AI applications, providing an excellent balance between computational performance and power usage, making it suitable for battery-powered applications. This work also provides a roadmap for engineers and researchers aiming to build intelligent standalone robotic systems on resource-constrained hardware. The study mainly focuses on deploying neural networks in microcontrollers, using the dual-core architecture of the RP2040 microcontroller to separate robot control from neural inference tasks.

Keywords: *microcontroller, neural networks, robotics control, edge AI, RP2040, TinyML, LiteRT*

Introduction: The combination of robotics on resource-constrained embedded systems and AI methodologies represents a significant advancement in autonomous systems design. The deployment of neural networks on microcontrollers enables the deployment of intelligent devices that require real-time decision-making capabilities without reliance on cloud connectivity. The shift toward edge AI has been particularly significant in robotics applications where low latency and power efficiency are important requirements. By focusing on edge computing, we can better meet the demands of modern applications that require quick data processing [1, 2]. The Raspberry Pi Pico, which uses the RP2040 microcontroller, provides a unique platform for investigating the practical implementation of neural network-based robotics control systems. The RP2040 microcontroller offers dual-core ARM Cortex-M0+ architecture operating at 133 MHz CPU and 264 KB of SRAM. These computational resources are sufficient to allow execution of sophisticated control algorithms while maintaining the power efficiency characteristics essential for embedded applications. The Raspberry Pi Pico is a handy microcontroller board that can be utilized in a variety of innovative applications, such as developing cost-effective solutions for real-time water quality monitoring [3] and low-cost educational platforms for teaching complex engineering concepts [4]. The adoption of LiteRT technology, formerly known as TensorFlow Lite, enables efficient neural network inference by providing a runtime environment specifically optimized for memory-constrained hardware platforms [5]. Current research in TinyML demonstrates that microcontrollers can achieve remarkable performance in machine learning tasks through careful

optimization techniques, including quantization and model compression. The challenge lies in connecting AI model development and practical deployment on embedded systems while maintaining acceptable accuracy levels and real-time performance constraints [6]. The integration of neural networks with robotics control systems on microcontrollers requires addressing multiple technical considerations, including memory limitations and computational bottlenecks [7].

The field of TinyML has experienced rapid growth as researchers and engineers search for methods to deploy machine learning algorithms on resource-constrained devices. Studies have demonstrated that fully connected neural networks can be successfully implemented on widely available microcontrollers with careful consideration of memory and computational constraints. Recent benchmarking efforts have evaluated various neural network architectures across different microcontroller platforms, revealing that performance considerably varies based on the specific hardware characteristics and optimization strategies [8]. Lower-precision quantized neural networks, such as 8-bit and 4-bit variants, can maintain acceptable accuracy while reducing memory requirements. Advanced quantization techniques, including mixed-precision approaches, allow developers to optimize different layers of neural networks based on their sensitivity to precision reduction. The development of specialized frameworks such as QKeras and Larq has provided tools for making quantized neural networks that target low-power applications [9].

Traditional robotics control systems have relied on powerful computing platforms to execute complex algorithms. However, recent advances in embedded systems design have enabled the implementation of complex control algorithms on microcontrollers. The integration of computer vision and complex autonomous control algorithms with robotics systems has been demonstrated in applications such as agricultural robots and robot navigation systems. These implementations highlight the possibility of deploying neural network-based perception and complex control algorithms on embedded platforms while maintaining performance requirements [10, 11].

The RP2040 microcontroller features a dual-core ARM Cortex-M0+ architecture that provides computational advantages over single-core alternatives. Each core operates independently, allowing for parallel processing of multiple tasks without requiring complex scheduling approaches. The architecture enables efficient communication between cores, while the programmable I/O subsystem provides flexibility for interfacing with various robotic sensors and actuators. Studies have demonstrated the RP2040's effectiveness in robotics applications, including precision control systems and intelligent robotic platforms. For example, the Scribe AI system uses the RP2040 microcontroller to deliver an affordable robotic handwriting solution with impressive precision [12]. The microcontroller's ability to handle real-time control tasks while simultaneously processing sensor data makes it particularly suitable for autonomous robotic systems. RP2040 can also be a good choice for battery-powered robotic applications considering its low operational power requirements.

Robot Hardware Design and System Architecture

The robot is designed for obstacle avoidance and is built on a simple two-wheeled drive platform with a chassis that provides a stable foundation for all components. The movement is controlled by two DC motors attached to a wheel and driven by a dedicated L293D motor driver module connected to the Raspberry Pi Pico microcontroller development board. Power is supplied by a rechargeable battery with a booster module for producing 5V output, providing a portable energy source. For sensing the environment, the robot uses an HC-SR04 ultrasonic sensor module mounted on the servo motor for adjusting the scanning angle. This sensor measures the distance to objects by emitting an ultrasonic pulse and calculating the time it takes for the echo to return. A breadboard is used to connect all the components to the microcontroller board, allowing for easy prototyping and testing of electronic circuits without the need for soldering (Figure 1). The Raspberry Pi Pico board serves as the brain, processing the sensor data and controlling the motor driver based on the output of the deployed neural network model.

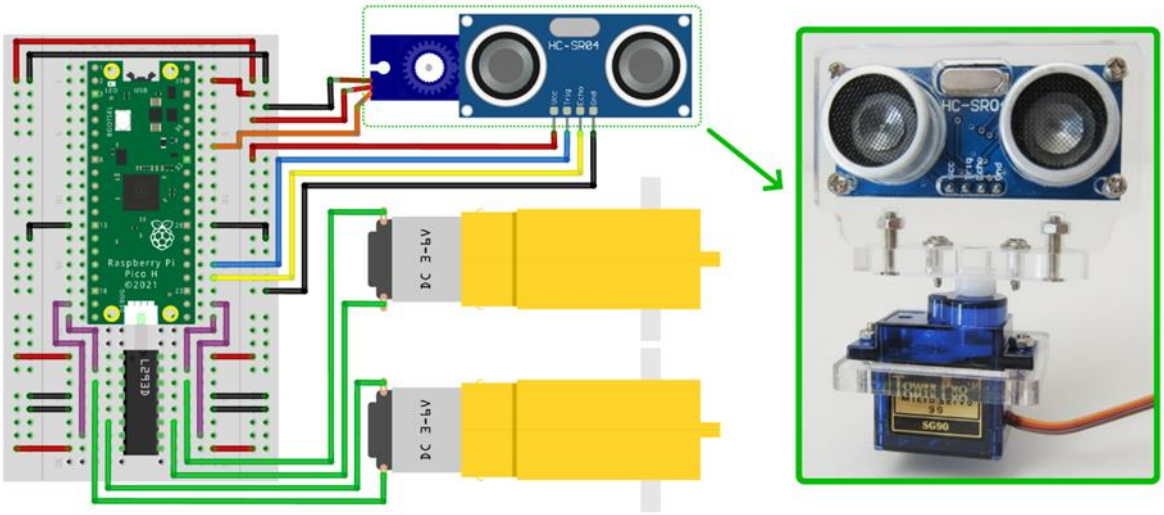


Figure 1. Breadboard setup for robot assembly with an ultrasonic sensor mounted on a servo motor

The RP2040-based system architecture uses the dual-core ARM Cortex-M0+ configuration to implement a distributed processing approach for neural network inference and robotics control. The first core is dedicated to executing the primary control loop and managing data processing, while the second core handles neural network inference tasks using the LiteRT runtime environment. This separation ensures that critical real-time control operations are not disrupted by the computational demands of neural network processing. Efficient memory management is also important for successful neural network deployment on the RP2040 platform. The memory architecture utilizes the 264 KB of on-chip SRAM by partitioning it between control system variables and neural network processing. The system uses SRAM for active model parameters and external flash storage for model weights that are not immediately required.

The implementation process begins with neural network training on desktop platforms using the TensorFlow framework. The trained models go through a systematic optimization pipeline that includes quantization to reduce precision requirements. Post-training quantization techniques enable the conversion of floating-point neural network models to integer representations without significant accuracy degradation. The quantization process reduces model size while maintaining acceptable inference accuracy for most robotics control tasks [13, 14]. The optimized models are then converted to LiteRT format for microcontroller deployment. The LiteRT framework provides essential capabilities for deploying neural networks on the RP2040 platform. The runtime environment implements several key optimizations that significantly improve inference performance on resource-constrained hardware. These optimizations are particularly important for maintaining real-time performance requirements in robotics applications. The development workflow also incorporates automated testing procedures to validate model performance throughout the process. Each model is evaluated against test datasets to ensure that accuracy degradation remains within acceptable limits. The workflow also includes real-time simulation with physical hardware to verify performance on the RP2040 platform.

Data Collection and Model Training

The neural network is designed to be highly efficient, with minimal layers and parameters to ensure it can run on the RP2040 microcontroller. The model was trained offline on a dataset of obstacle distances and corresponding safe maneuvers. The training process begins with data collection by a human operator who manually controls the robot in various environments, recording a dataset of obstacle distances and corresponding safe actions. For example, if an object is detected at a short distance directly in front of the robot, the corresponding action would be to turn left or right. If the path is clear, the action would be to go straight. This process builds a training dataset that maps sensor input to desired robot behavior. Synthetic data generation supplements this by creating additional, labeled examples of obstacle scenarios, significantly expanding the dataset without the need for manual data collection. This technique allows handling challenging scenarios

that may be difficult to capture in the real world, improving the model's flexibility. The final dataset is used to train a simple feedforward neural network for the obstacle-avoiding wheeled robot system. The training process involves feeding the data into the network and adjusting its internal weights and biases to minimize a loss function. This function measures the difference between the network's predicted actions and the correct actions from the dataset. During training, the mean squared error is used as a loss function to quantify the difference between the model's predicted values (ŷi) and the actual values (yi) for a given batch of N data points. The goal of the training process is to improve the model's accuracy by reducing this error, as minimizing the LMSE helps the model learn to make more accurate predictions by penalizing larger errors more heavily.

$$L_{MSE} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2 \quad (1)$$

The deployment process begins with converting the trained model into a format compatible with the LiteRT runtime environment. This step also involves quantization, where the model's parameters are reduced from floating-point numbers to lower-precision integers. This technique significantly reduces the model's size and computational requirements without a substantial loss in accuracy, making it suitable for the limited memory and processing power of the RP2040. Once converted, the model is transferred onto the RP2040 microcontroller's flash storage. A custom firmware written in C is then used to interface with the LiteRT runtime. This firmware includes the core logic and reads data from the ultrasonic sensor to feed it as input to the deployed model via the LiteRT API and then uses the model's output to control the motor driver. The LiteRT runtime is highly optimized for fast inference and efficient memory usage on the RP2040. The model inference and control procedures must be fast enough to achieve real-time obstacle avoidance. This allows the robot to make quick, intelligent decisions in real-world scenarios. After deployment, the model is evaluated to ensure it has learned to generalize beyond the training data and can make accurate predictions in new scenarios. The inference process of a neural network with multiple layers can be represented by a recursive function, where the output of each layer is calculated from the output of the previous layer.

$$a_l = \sigma_l(a_{l-1}W_l + b_l) \quad (2)$$

Here, Wl and bl are the weight matrix and bias vector for layer l, which are learned during the training phase. The function σl is the non-linear activation function, such as ReLU or Sigmoid. The resulting model's purpose is to take the distance data from different angles using the ultrasonic sensor as input and output the appropriate motor control signals. The network's input layer has 5 neurons to accept the ultrasonic sensor distance measurements from different angles. The output layer has neurons corresponding to the possible actions, such as turning left, turning right, and going straight. The neural network also includes two hidden layers between the input and output layers. Each of these hidden layers contains 16 neurons, which enables the model to process complex relationships between the raw sensor readings and the appropriate motor control commands (Figure 2).

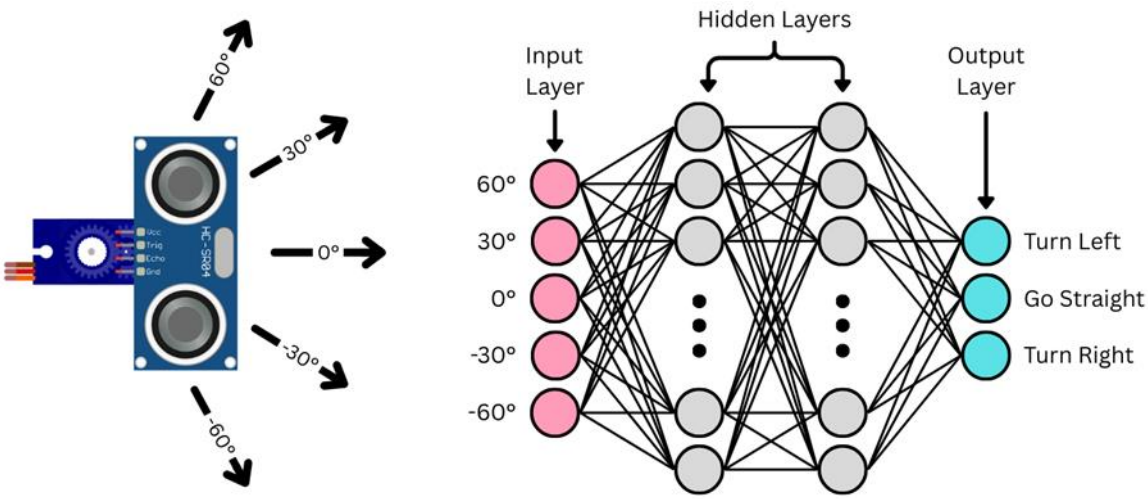


Figure 2. The structure of the neural network model that uses distance measurements from an ultrasonic sensor to determine motor control actions for robot navigation

Experimental Results and Analysis

Experimental evaluation of the RP2040-based neural network system demonstrates that the inference times for tested neural networks range from 30 to 1000 milliseconds, depending on model complexity and characteristics. These performance characteristics are sufficient for most robotics applications. Power consumption analysis reveals that the RP2040 microcontroller achieves remarkable energy efficiency, with total power consumption remaining below 200 milliwatts during active operation. The RP2040 microcontroller also enables power management strategies that can reduce consumption to less than 5 milliwatts during idle periods while maintaining system responsiveness. These power characteristics make the system suitable for battery-powered autonomous robotics applications.

Quantization experiments demonstrate that 8-bit integer representations maintain acceptable control accuracy for required navigation tasks. Mixed-precision quantization strategies can improve the performance by allowing critical control parameters to retain 16-bit precision while less sensitive computations use 8-bit or even 4-bit representations. Testing under various operating conditions, including temperature variations and supply voltage fluctuations, shows that the system maintains stable performance across the full operating range. The neural network-based control algorithms exhibit greater adaptability compared to traditional fixed-parameter controllers, particularly in dynamic environments with changing conditions. Memory utilization analysis shows that optimized neural networks typically require between 50 and 150 KB of the available 264 KB SRAM, depending on model complexity. This leaves sufficient memory for system variables and sensor data buffers while enabling the implementation of relatively sophisticated neural architectures. Model compression techniques can further reduce memory requirements with minimal impact on model accuracy while increasing inference performance.

The RP2040 microcontroller provides several distinct advantages for neural network-based robotics control applications. The dual-core architecture enables true parallel processing of control and inference tasks without the complexity associated with multi-threaded implementations on single-core systems. The relatively large memory capacity compared to other microcontrollers in its class provides sufficient space for implementing sophisticated neural architectures. The programmable I/O system also offers unique flexibility for interfacing using miscellaneous communication protocols commonly found in robotics applications. This flexibility reduces the need for external interface circuits and simplifies system integration. However, despite its advantages, the RP2040 platform presents several challenges for neural network deployment. The ARM Cortex-M0+ cores lack dedicated floating-point units, requiring software emulation for floating-point operations. This limitation can impact performance for applications requiring high-precision arithmetic operations. However, the availability of floating-point support through fast software libraries located in ROM ensures that high-precision computations can still be performed

when necessary with minimal impact on system performance. Memory and flash storage size constraints can also become limiting factors for larger neural networks. Additionally, the lack of hardware acceleration for neural network operations means that all computations must be performed through general-purpose arithmetic units.

Conclusion:

This study demonstrates the successful implementation of neural networks for robotics control on the RP2040 microcontroller platform using LiteRT technology. The experimental results confirm that sophisticated AI-powered control systems can be deployed on resource-constrained embedded platforms while maintaining real-time performance and energy efficiency requirements. The dual-core ARM Cortex-M0+ architecture provides sufficient computational capacity for executing optimized neural networks alongside traditional control algorithms. The integration of LiteRT enables significant performance improvements through specialized optimizations. The achieved inference times of 30-1000 milliseconds with power consumption below 200 milliwatts demonstrate the viability of the approach for practical robotics applications. The memory utilization analysis shows that models requiring 50-150 KB of memory can be successfully deployed while maintaining system responsiveness and functionality.

Future research directions include the development of more sophisticated neural architecture algorithms specifically designed with microcontroller constraints in mind. These algorithms could automatically generate optimal network architectures that balance accuracy and computational requirements for specific robotics tasks. The integration of learning capabilities could also enable systems to adapt to changing environmental conditions and improve performance over time. Advanced quantization techniques such as non-uniform quantization represent promising areas for reducing memory requirements while maintaining accuracy. The development of specialized hardware accelerators for common neural network operations could significantly improve performance on future microcontroller platforms.

References

1. W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge Computing: Vision and Challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637-646, Oct. 2016, https://doi.org/10.1109/JIOT.2016.2579198

2. M. Rybczak, N. Popowniak, and A. Lazarowska, "A Survey of Machine Learning Approaches for Mobile Robot Control," *Robotics*, vol. 13, no. 1, p. 12, Jan. 2024, https://doi.org/10.3390/robotics13010012

3. M. J. E. Fortes et al., "The Utilization of Date (Phoenix dactylifera) Pits and Raspberry Pi Pico in Making a Water Quality Management Device," *International Journal of Innovative Science and Research Technology (IJISRT)*, vol. 10, no. 4, pp. 2208-2213, Apr. 2025, https://doi.org/10.38124/ijisrt/25apr1334

4. S. Ojeda-Mancera et al., "Didactic Hardware in the Loop Platform: A Low-Cost Open-Source Approach," *IEEE Latin America Transactions*, vol. 23, no. 10, pp. 910-921, Oct. 2025, https://doi.org/10.1109/TLA.2025.11150634

5. V. Krasteva, T. Stoyanov, and I. Jekova, "Implementing Deep Neural Networks on ARM-Based Microcontrollers: Application for Ventricular Fibrillation Detection," *Applied Sciences*, vol. 15, no. 4, p. 1965, Feb. 2025, https://doi.org/10.3390/app15041965

6. B. Sudharsan et al., "TinyML Benchmark: Executing Fully Connected Neural Networks on Commodity Microcontrollers," in *2021 IEEE 7th World Forum on Internet of Things (WF-IoT)*, 2021, pp. 883-884, https://doi.org/10.1109/WF-IoT51360.2021.9595024

7. A. Mujtaba, W.-K. Lee, and S. O. Hwang, "Low Latency Implementations of CNN for Resource-Constrained IoT Devices," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 69, no. 12, pp. 5124-5128, Dec. 2022, https://doi.org/10.1109/TCSII.2022.3205029

8. R. Wainbuch and A. J. Samuel, "TinyML: Deploying Machine Learning on Microcontrollers for IoT Applications," *Journal of Science, Technology and Engineering Research*, vol. 2, no. 2, pp. 44-57, Dec. 2024. https://doi.org/10.64206/d8sh8k34

9. D. Pau, M. Lattuada, F. Loro, A. De Vita, and G. Domenico Licciardo, "Comparing Industry Frameworks with Deeply Quantized Neural Networks on Microcontrollers," in *2021 IEEE International Conference on Consumer Electronics (ICCE)*, 2021, pp. 1-6, https://doi.org/10.1109/ICCE50685.2021.9427638

10. M. S. Benyeogor et al., "An Algorithmic Approach to Adapting Edge-based Devices for Autonomous Robotic Navigation," *EAI Endorsed Transactions on Context-aware Systems and Applications*, vol. 8, no. 1, Aug. 2021, https://doi.org/10.4108/eai.5-8-2021.170559

11. E. Kayacan, Z. Zhang, and G. Chowdhary, "Embedded High Precision Control and Corn Stand Counting Algorithms for an Ultra-Compact 3D Printed Field Robot," in *Robotics: Science and Systems*, 2018, https://doi.org/10.15607/RSS.2018.XIV.036

12. R. A. Xiong, "Cost-Effective Design of Scribe AI – Robotic Handwriting Systems Using Raspberry Pi Pico and 3D Printing," in *2024 IEEE Long Island Systems, Applications and Technology Conference (LISAT)*, 2024, pp. 1-6, https://doi.org/10.1109/LISAT63094.2024.10807994

13. A. Julião, G. Rombo, and J. P. Oliveira, "Comparative Study of Quantized CNN Inference on ARM and RISC-V Microcontrollers," in *2025 9th International Young Engineers Forum on Electrical and Computer Engineering (YEF-ECE)*, 2025, pp. 62-66, https://doi.org/10.1109/YEF-ECE66503.2025.11117526

14. P.-E. Novac, G. Boukli Hacene, A. Pegatoquet, B. Miramond, and V. Gripon, "Quantization and Deployment of Deep Neural Networks on Microcontrollers," *Sensors*, vol. 21, no. 9, p. 2984, Apr. 2021, https://doi.org/10.3390/s21092984